MMA3001 Individual Project Report – 2D Panel Method

Ryan Heskey – 33858209

25 October, 2026

Project GitHub: https://github.com/ryanheskey717-blip/2d-panel-method

1.0 Introduction

The aim of this project was to predict the 2-dimensional aerodynamics around an arbitrary aerofoil section using the potential-flow panel method, while also learning how to organise and undertake a professional coding project. This method involves setting up 'panels' on the aerofoil surface which then have elementary potential flows placed on them which all work together to influence the overall flow. With the correct boundary conditions constraining the system, the linear system can be solved for the required elementary flow strength of each panel. The scope for this project included fully understanding and implementing the linear system that solves for the strengths, and post processing these results to arrive at aerodynamic coefficients. This scope was set to have an optional extension of investigating the implementation of viscous boundary layer and turbulence corrections to improve the results. The engineering problem that is being solved here is the issue of quickly characterising the aerodynamics of an arbitrary aerofoil shape in different conditions. Being able to calculate a medium-fidelity estimate of these values very quickly, without full Computational Fluid Dynamics (CFD), is very important to the design of new wings and aerodynamic surfaces.

2.0 Method

2.1 Inputs and Outputs

The inputs to this code are quite simple, consisting of the aerodynamic conditions at which the solution is found, and the shape of the aerofoil in question. Both are expressed in a .yaml input file, with the shape defined in a .dat or .txt file as an array of coordinates. These can be written manually or copied from a database of existing aerofoils [1], [2]. Within the code, these coordinates are loaded, and some important geometry parameters are calculated, such as the normal vectors. The geometry is a very important input into the matrices for the linear system, and the freestream velocity is the other most important for this part of the process. The outputs of the linear system are fed into the rest of the code to calculate aerodynamic forces and moment, and these are the main output of the system. When ran in sweep mode, the code saves a .csv file of the forces, moment and coefficients across all tested angles of attack. The code also can load these files and create plots over angle of attack and compare to downloaded XFOIL data from [3], which is the industry standard for 2D aerofoil section aerodynamics. The results and validation of these outputs are discussed in Section 3.

2.2 Linear System for Inviscid Aerodynamics

There are a few different ways to go about setting up and solving panel methods. The chosen method for this project is known as the Hess-Smith panel method. In this method, a constant strength source distribution and a constant strength vortex sheet are placed over the panel surface. The velocity contribution at a point is found by integrating an elementary source and vortex [4] over the length of the panel, which gives the distributions. Each panel is allowed to have a different source strength, but all panels have the same vortex strength. It is formulated like this so that the source strengths are found by enforcing the no penetration boundary condition at all control points, which are just the centres of each panel. This leaves just the vortex strength to be solved for. To do this, the Kutta condition is enforced at the trailing edge, which closes the system and allows all strengths to be found. This is a linear system due to the

assumption of potential flow. When a flow is assumed to be inviscid and irrotational, the Navier-Stokes Equations become linear. This means that individual solutions can be added together to form a more complex but still valid solution. The system here is a $(N + 1) \times (N + 1)$ system, where N is the number of panels and the extra equation and unknown is the Kutta condition and vortex strength. The system is set up according to the matrix equation $A.x = b$, where A encodes the influence that each panel has on the velocity at every control point, x is the strengths to be solved for, and b depends on the panel normal and freestream velocity and ensures that the Kutta condition is met.

2.3 Viscous Boundary Layer Extension

As this was an extension to the original scope, there was not a large amount of consideration taken in this section. The method used in the code was to implement a laminar flat plate Blasius solution no matter of the actual geometry. This proved to be quite unphysical, as will be seen in Section 3, and there were some issues with the implementation, but this was not able to be fixed in time, so there were no further methods used here. Despite the lack of good results for this, the method of implementing this coupling was quite interesting, and the order of operations had to be very well thought out. In the end, an iterative method was used, where the inviscid system was solved, then this information informed, or would have informed, the proper viscous solver, updating the effective aerodynamic body. The effective aerodynamic body, as defined by the displacement thickness, was accounted for in the inviscid system using the transpiration method, which is also used in XFOIL [5]. This process would then be repeated until convergence was reached. Due to the lack of a viscous system that was dependent on the inviscid flow, this full method was not able to be fully tested to convergence as the potential flow and Blasius solution were fully defined and would not change after the second iteration.

2.4 Post Processing

The other important part of the computation was the post processing of the calculated fluid flow. The main part of this included the pressure calculation which was then used to calculate the total forces and moment. Within the inviscid flow, the Bernoulli method of calculating pressure is valid, so this was used. It was assumed that the boundary layer had no pressure gradient in the normal direction, so the inviscid pressure was imprinted on the actual aerofoil's surface. To find the total force and moment, the dot product of the pressure and skin friction coefficients with the normal and tangential directions were numerically integrated. The chosen method for this was the midpoint method as it was the simplest from a code point of view. Despite not being the most accurate method, it handled the complex form of equations with the dot product well and was not the largest source of error due to the flow solver having more stringent mesh fineness criteria. It was also chosen as it is very fast and the entire benefit of the panel method over CFD is how fast it runs.

3.0 Results and Validation

The code also contains support of visualising results, which is perhaps the best way to see how well the code is working. In a visualisation of the fluid flow, it can be seen whether the no penetration boundary condition and the Kutta condition are actually being satisfied. It is also known what the flow around an aerofoil should look like, for example it is well known that the top surface of a lifting aerofoil should have a lower pressure than the bottom surface. Two examples of these visualisations are shown in Figure 1 and 2, for different conditions and aerofoil shapes. The streamlines can be seen to never cross the surface which shows that the no penetration boundary condition is being met. Both of these examples should be producing lift, and the top plot shows that the pressure is indeed higher underneath the aerofoil. The second plot shows a faster velocity over the top of the aerofoil which corresponds to a lower pressure, meaning that this is also behaving as expected. The Kutta condition requires that the

flow leaves the trailing edge tangentially, and this can be seen at both trailing edges, where the flow is continually smooth.

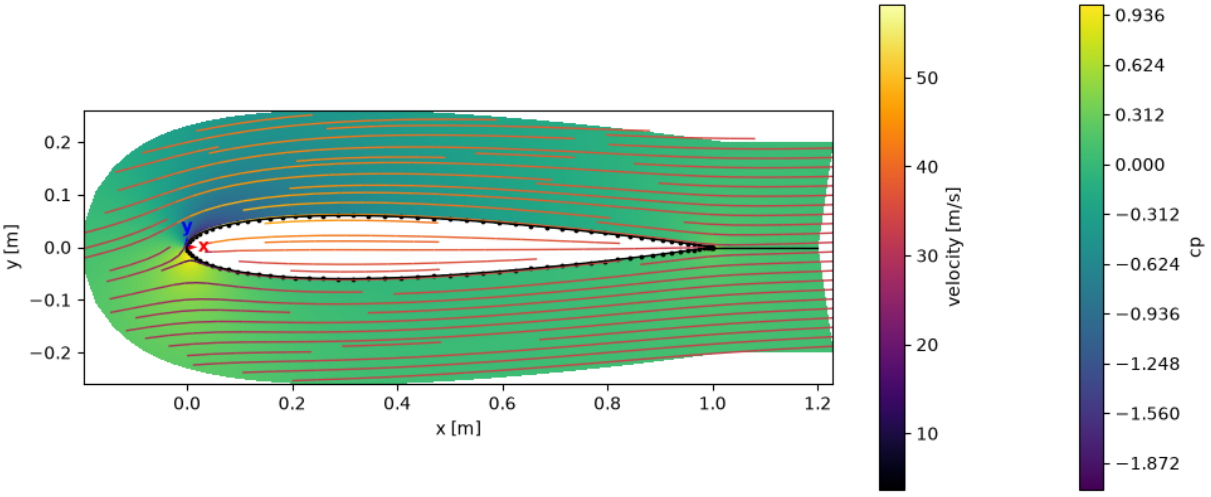


Figure 1: Flow around a NACA0012 aerofoil at an angle of attack of 5°. Streamlines are coloured by the local velocity, and the background shows the calculated pressure field.

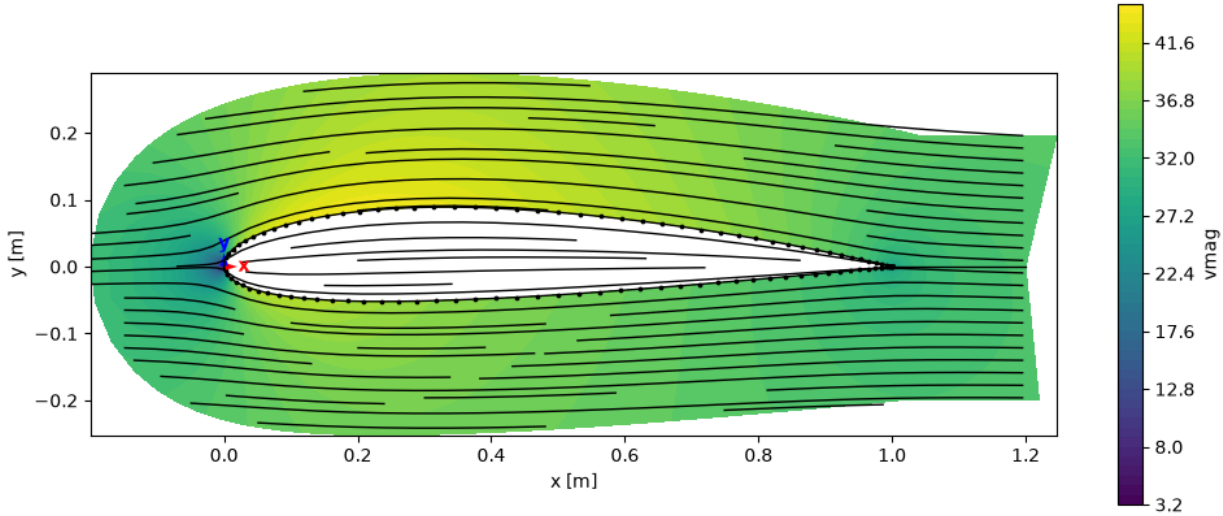


Figure 2: Flow around a NACA2414 aerofoil at an angle of attack of 0°. Background contours show the calculated velocity magnitude field, with black streamlines.

Of course, this only provides validation of the inviscid flow field. There was limited validation done on the viscous counterpart as there were issues with the implementation, as discussed in the last section. To validate the pressure calculation, the generated data needs to be compared to known data. This needs more than this flow visualisation. To undertake this comparison, the code was modified to include the ability to load and compare XFOIL data. XFOIL [5] is the industry standard for aerofoil section analysis and is used by Airfoil Tools [1]. This part of the code includes the option to plot the calculated results and the XFOIL results on the same figure. This allows the direct comparison of important results to what they should be. The following three figures show some interesting results.

XFOIL [5] is the industry standard for aerofoil aerodynamic analysis. It is also a panel code with viscous-inviscid coupling. It has been validated over many years since it was first developed in 1986. It is used widely for aerofoil databases such as Airfoil Tools, and most plots of an aerofoils performance are likely generated by it. This program has provided some inspiration for this project, especially in the method of viscous coupling, utilising the

transpiration method. This is a very accurate program and serves as a very good baseline to test this implementation with. In practice, XFOIL would be used to solve this engineering problem instead of developing a new program due to its robustness, validation and widespread use.
Figure 3 shows the results for a NACA0012 aerofoil over a range of angles of attack, and it can be seen that the lift is incredibly well predicted at lower angles of attack. This proves that the chosen method of integrating pressure to get the total force is working correctly. It is very important to note that this has the viscous coupling turned off, so this is purely an inviscid panel method solution. This explain why drag is not predicted, and why it cannot predict flow separation and stall like XFOIL can. The predicted moment is interesting; it is hard to tell where it is good or not. Obviously the more complex behaviour is not predicted, but the overall trend is correct.

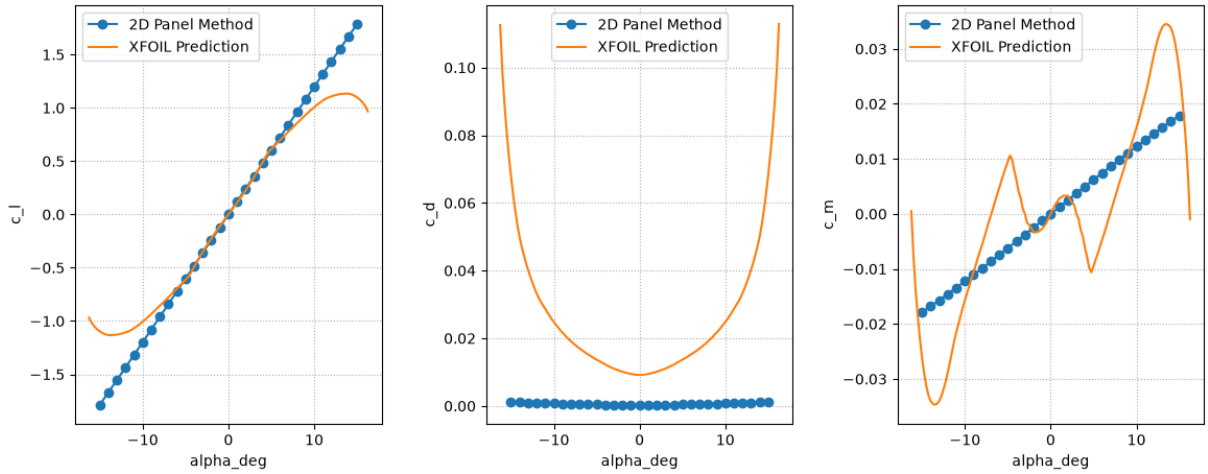


Figure 3: NACA0012 results, with viscous coupling inactive. (Re = 200000)

With the exact same conditions, and the only change being the inclusion of the viscous coupling, the results can be seen to be quite different in Figure 2. Again, the lift slope is well predicted, with only very small oscillations being observed. The drag and moment predictions, however, look quite horrible. Large oscillations are seen that are obviously unrealistic, and this means that there is an issue with the viscous solver. From a small investigation, it was found that this issue is likely caused by a bad method of calculating and using the stagnation point near the leading edge of the aerofoil, but there was not enough time in the project to attempt to rectify this issue. This issue does exemplify how well little the viscous effects change the lift, at least until separation occurs, showing why inviscid methods like the thin aerofoil theory can work very well for lift.

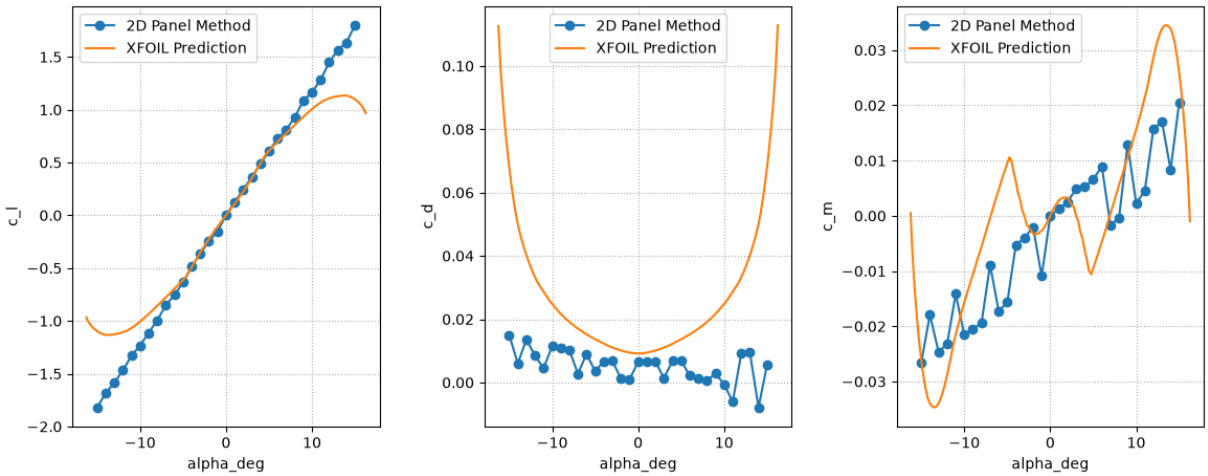


Figure 4: NACA0012 results, with viscous coupling active. (Re = 200000)

The other important case to check is what happens with a cambered aerofoil. Looking at Figure 5 below, it can again be seen that the lift is well predicted for the NACA2414 aerofoil. This aerofoil has a non-zero angle of zero lift, which is also predicted correctly by this panel method. Again, with the lack of viscous coupling, the drag is not predicted properly. The moment seems to be doing something weird, as it is neither near the correct value or following the trend like it was in the symmetric case above. It is possible that something is wrong with the moment calculation, but more investigation would be necessary to figure out why it is so wrong. Again, the viscous coupling returned no useful results so was that comparison was omitted from this report.

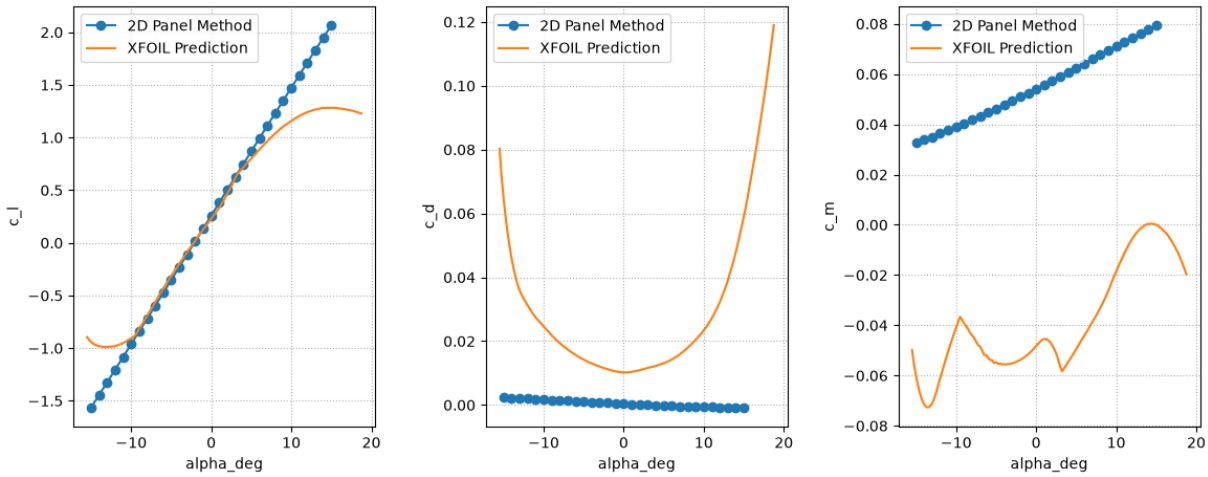


Figure 5: NACA2414 results, with viscous coupling inactive. (Re = 200000)

Overall, it was found that the 2D panel code in its current state can only be ran with no viscous coupling to get smooth results. The downside of this is that drag cannot be predicted at all, however it has been seen that the lift is well predicted by the inviscid flow field for some conditions. With these results, it can be said that this code solves the engineering problem of quickly characterising the aerodynamic performance of an arbitrary aerofoil in a very limited capacity. It can accurately predict the lift slope and the zero-lift angle of attack, which are very useful at low angles of attack, but it fails to predict viscous phenomena such as stall and drag at all. With more refinement of the viscous methods, and an investigation into the issue with the moment, this code has the potential to fully solve the problem.

4.0 Optimisation

Optimisation of this code is important, as the main benefit of panel methods over full CFD is the large computation time reductions that are possible. Unfortunately, there is not too much room for optimisation in the internal workings of the panel method due to the way that the calculations are performed. The bulk of the time spent on running the code is taken up by calculating the components of the large matrix A. Each component of this $(N + 1) \times (N + 1)$ matrix has to perform a calculation of what the velocity would be for a source strength of one, meaning that this has to be performed ~N^2 times, as there are a few terms that don't actually require this. Fortunately, the entirety of A is not dependent on any of the conditions. Since it is only a function of the geometry, and this does not change from case to case in a sweep over different conditions. This is taken advantage of by passing the matrix that was calculated in the first iteration all the way through each case, without needing to recalculate the entire linear system. This optimisation resulted in each case running about twice as quickly as before which is a huge improvement. This is also possible within a single case when iteratively solving viscous and inviscid, because the transpiration velocity method only affects the right-hand-side, the b vector, of the inviscid linear system. Sweeps overwrite previous cases to ensure that

memory issues are not a problem, with each case saving important results to a .csv file at the end of its run.

Comparatively, the actual solving of the linear system is quite fast, which is just handled by NumPy's *linalg.solve(A, b)* which uses LU decomposition to quickly solve the system. This also has built in methods to help with numerical stability, such as pivots that swap lines around within the system. Another issue that panel methods sometimes run into is when control points are located exactly on the panels. To fix this, every control point is offset by a very small amount ($< 10^{-5}\ m$) to improve the numerical stability of the system. With more development time, a version of LU solving would have been used to directly pass the *L* and *U* matrices to the solver for the next case so they do not need to be created by the solver from *A* every time, which should help to further speed up this code. In its current format, the average case within a sweep takes about 0.16 seconds to run fully with *A* passed directly without needing to calculate it, for a mesh with 100 panels. A full sweep of 31 cases from -15° to 15° angle of attack with 1° increments takes 5.2 seconds.

5.0 Limitations and Learnings

As was seen in Section 3 Results and Validation above, there are a few limitations of the code. The most obvious of these is the corruption of results that the viscous coupling causes. This should be a relatively simple fix, but there is a lot that would need to be added to the viscous solving to make it capable of reproducing the results generated by XFOIL. This is a major limitation of this code and will prevent it from being used to accurately and reliably being to solve the engineering problem. Despite this, it can still predict parts of the aerofoil's aerodynamics that inviscid solutions excel at and could probably outperform thin aerofoil theory due to its ability to model the actual shape of the aerodynamic body, and not just the camber line. However, before the code could be used for even this, much more thorough analysis of how it perform over a wider range of conditions and aerofoil types needs to be conducted, to make sure there are no issues with other parts of the code, like what was seen with the moment in Figure 5.

There have been many learnings that have come with this project. The first and foremost of these being the amount of thought that is required to figure out the exact layout of the code for it to be able to do what it needs to. There were many instances through this project where I thought I had found the best method to do a specific thing, only to go to add the next part and realise that this way actually won't work at all. At a few points through the project I had to take a step back and write out the form that the code needs to take and make some major changes to better accommodate this. The most major of these was when I found out about the transpiration method, and I had to add the scaffolding to be able to do the iterative solving, and also changing the inviscid solver to accommodate a changing effective aerodynamic body. I found however, that these re-evaluations of the core structure of the code led to a much better end product that I can confidently say is the best way that I found to do what I wanted it to do. Without this, I would have clung to one way of doing things, and the code would not be nearly as capable as it is now.

Another major learning was how hard I had to try to avoid scope creep. Even though I set the viscous coupling as an optional extra that I could try and add if I got the initial inviscid part working well and well formatted, documented and tested, I found myself trying to include it before I got to this stage. While it was okay in the end because I was constantly keeping in mind how to add it, which made it a lot easier when I did get to including it, I can now see that I was focusing a bit too much on it. This left me with less time to properly document and test the code as I had added a lot more to it, and this has taught me that next time I need to be

stricter with how I set the scope and optional extras. Next time I undertake a project like this, I plan to write out exactly I want the minimum viable product to be and what state I'd like it to be in, and be very strict about not adding anything until it is past that point.

6.0 Artificial Intelligence Use

Throughout this project, I tried to avoid using Artificial Intelligence (AI) to actually write the code so that I would have the opportunity to actually understand each and every line in the project. Instead, AI was used to help explain concepts, suggest best practices and automate certain repetitive tasks. Due to the relatively large number of classes and functions in my project, AI was really useful in the documentation stage, where it helped to generate docstring headers for functions and classes based off the inputs and outputs of each function. Each output was slightly modified to match exactly what I wanted to communicate in this documentation, and any extra parts containing other necessary information was added. AI also helped to further my understanding of the concepts that I was learning about, providing examples of how it worked. The other way that AI helped with this project was debugging code. When I was implementing the elementary flows that make up the distributions spread over the panels, I kept running into issues of inputting the wrong variables or doing things in the wrong order, due to some large equations and the specific way that it needs to be done. After debugging by hand, I still couldn't get it working properly so I gave it to AI, and it suggested some errors that it could see. After a bit of trial and error with its solutions, I managed to get it working properly. Using AI as a tool in this way definitely reduced the amount of time that I spent of fixing small issues and allowed me to create a better overall product.

In this project, I made a point of never blindly trusting AI with anything. I never let it make any decisions, and I only used the information it provided to inform decisions that made sense to me. In some cases, AI provided me with some formulas that weren't the correct ones for what I was trying to do. I was immediately sceptical as they didn't quite make sense to me, so I double checked my references and sure enough I found the correct ones this way. I also tested a different AI model to see if it would make the same mistake, and that one told me the correct answer. This exemplifies why AI should not be trusted in its current state, as it is not always correct and could provide a different answer for the same question. This further supported my decision to check absolutely everything before using it. I feel that this approach allowed me to have a much better understanding of the project and end up with a much better final product.

7.0 Conclusion

The aim of this project was to predict the 2-dimensional aerodynamics around an arbitrary aerofoil section, to solve the engineering problem of characterising the performance of a new aerofoil shape much faster than full CFD. Using panel methods, the inviscid flow field around the aerofoil was solved accurately, providing good results for the lift at low angles of attack. The viscous coupling had some issues with its implementation which resulted in non-physical phenomena seen in the drag and moment results. This performance was compared to the industry standard for aerofoil analysis, XFOIL, which is also a coupled inviscid-viscous panel method solver that has been validated over a very long time. Some optimisation of the code was conducted with some major speedups being achieved for sweeps over angles of attack. AI was used in small amounts for repetitive tasks and explanations, and all outputs were checked thoroughly for accuracy. The limitations of this code prevent it from fully solving the engineering problem, but the working inviscid part of the code is accurate and models the flow around and the lift of a thick aerofoil well. The other goal of this project was to learn how to properly undertake a professional coding project, and this was achieved well through lots of important learnings and takeaways that will be taken forward into future engineering projects.

8.0 References

1. Airfoil Tools. *NACA 4 digit airfoil generator.* 2026. Accessed October 25, 2026. http://airfoiltools.com/airfoil/naca4digit?MNaca4DigitForm%5Bcamber%5D

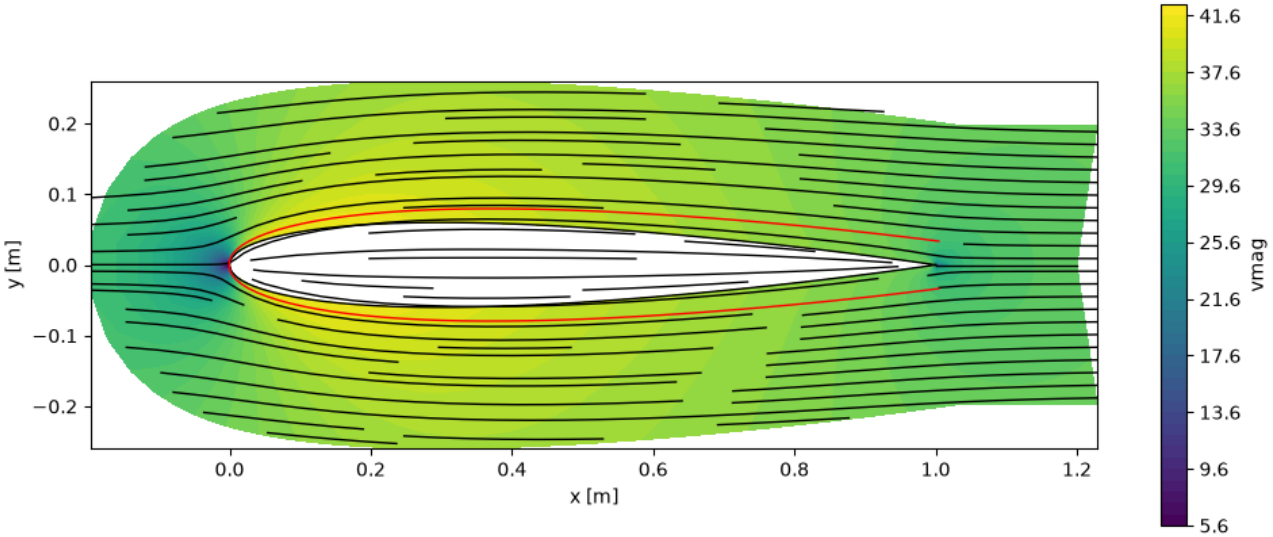
2. University of Illinois at Urbana-Champaign – Applied Aerodynamics Group. *UIUC Airfoil Coordinates Database*. 2026. Accessed October 25, 2026. https://m-selig.ae.illinois.edu/ads/coord_database.html

3. Airfoil Tools. *NACA 0012 AIRFOILS (n0012-il) Xfoil prediction polar at RE=200,000 Ncrit=5.* 2026. Accessed October 25, 2026. http://airfoiltools.com/polar/details?polar=xf-n0012-il-200000-n5

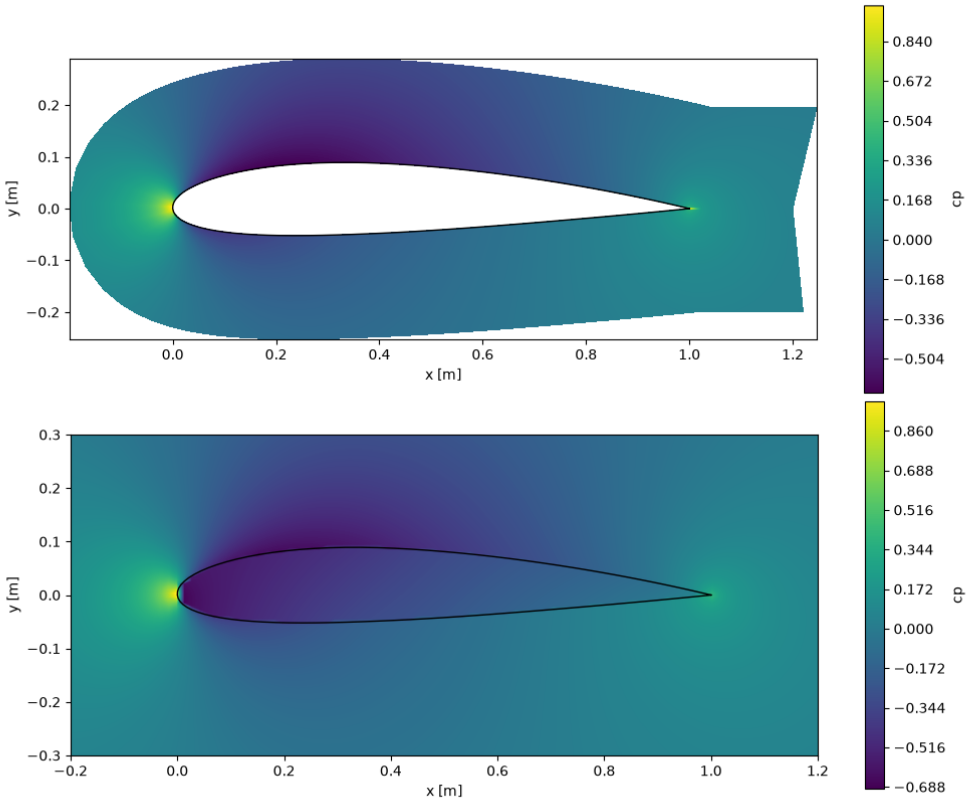
4. Anderson, JD. Jr. *Fundamentals of Aerodynamics.* 7th ed. McGraw-Hill; 2024

5. Drela M, Youngren H. *XFOIL – Subsonic Airfoil Development System.* 2026. Accessed October 25, 2026. https://web.mit.edu/drela/Public/web/xfoil/

9.0 Appendices



Appendix 1: NACA0012 with viscous coupling solution. Displacement thickness displayed with a red line. This has been exaggerated for the effects to be seen. Can be seen that the no penetration boundary condition is now satisfied on the edge of the boundary layer instead of the aerofoil surface due to the transpiration velocity which ‘pushes’ outward or ‘sucks’ in.



Appendix 2: Comparison of the different grid types used for off-body visualisation. Conformal (top) is much clearer that the cartesian (bottom) version and achieves a better resolution through weighted grid points, with finer spacings near the body.